חלק א'(40%)

1

ABH,ACDFGH,ACFGH

2

1: not word.startswith(curr)
2: curr[:-1]
3: len(prefixes)==1
4: prefixes.pop()

3

O(nlog(n))

4

O(n^2 * log(n))

5

lambda x : lambda *args: sum(map(x,args))

6

2**n-1

7

n<10
(n//10)%10
high_low(n//10)

8

x+y-1

9

[0,0]
16

10

e1 is e2
e1,e2
(min(e1[0],e2[0]),max(e1[1],e2[1]))

11

```
def find_path(domlist):
    return max((path_helper(domlist[:i]+domlist[i+1:],[domlist[i]])
                for i in range(len(domlist))),
               default=[],
               key=len)

def path_helper(domlist,candidate):
    return max([],candidate,
               *(path_helper(domlist[:i]+domlist[i+1:],candidate+[domlist[i]])
                 for i in range(len(domlist))
```

```
            if candidate[-1][1]==domlist[i][0]),
          key=len)
```

12

```
def count_unival_subtrees(root):
    return count_helper(root)[0]


def count_helper(root):
    if root is None:
        return 0,True

    left=count_helper(root.left)
    right=count_helper(root.right)

    unival = (left[1] and (not root.left or root.val==root.left.val) and
              right[1] and (not root.right or root.val==root.right.val))

    return left[0]+right[0]+unival,unival
```

13

```
class Item:
    allitems = {}

    def __init__(self, name, price, sale_price=None):
        self.name=name
        self.price=price
        self.sale_price=sale_price
        Item.allitems[name]=self

    def get_cost(self,quantity):
        if self.sale_price is None:
            return quantity*self.price
        return (quantity//self.sale_price[0]*self.sale_price[1]+
                quantity%self.sale_price[0]*self.price)

    def get_savings(self,quantity):
        return quantity*self.price-self.get_cost(quantity)

class ShoppingCart:
```

```
    def __init__(self):
        self.items={}

    def add_item(self, name:str, quantity:int):
        if name not in Item.allitems:
            raise ValueError('item does not exist')
        if name in self.items:
            raise ValueError('item already in cart')
        self.items[name]=quantity

    def remove_item(self, name:str):
        if name not in self.items:
            raise ValueError('item not in cart')
        self.items.pop(name)

    def get_total(self):
        return sum(Item.allitems[name].get_cost(quantity)
                   for name,quantity in self.items.items())

    def get_savings(self):
        return sum(Item.allitems[name].get_savings(quantity)
                   for name,quantity in self.items.items())

    def get_list(self):
        return list(self.items.items())
```



```
def unbalance_helper(root):
    if root.left is None and root.right is None:
        return root
    tail=root
    if root.left is not None:
        tail=unbalance_helper(root.left)
    tail.left=root.right
    if root.right is not None:
        tail=unbalance_helper(root.right)
    root.right=None
```

```
def palindrome_partition(s):
    if not s:
        return [[]]
    res = []
    for i in range(1,len(s)+1):
        if s[:i]==s[:i][::-1]:
            res+=[[s[:i]] + part for part in palindrome_partition(s[i:])]
    return res
```



```
    def update(self,user,likes):
        if user not in self.user_likes:
            self.user_rank[user]=len(self.user_likes)+1
            self.rank_user[len(self.user_likes)+1]=user
        self.user_likes[user]=likes

        rank=self.user_rank[user]
        while (len(self.user_likes)>rank and
               self.user_likes[user]<self.user_likes[self.rank_user[rank+1]]):
            self.user_rank[self.rank_user[rank+1]]=rank
            self.user_rank[user]=rank+1
            self.rank_user[rank]=self.rank_user[rank+1]
            self.rank_user[rank+1]=user
            rank+=1

        while (rank>1 and
               self.user_likes[user]>self.user_likes[self.rank_user[rank-1]]):
            self.user_rank[self.rank_user[rank-1]]=rank
            self.user_rank[user]=rank-1
            self.rank_user[rank]=self.rank_user[rank-1]
            self.rank_user[rank-1]=user
            rank-=1
```